Adversarial Search

EECS 492

Introduction to Artificial Intelligence

Reminders

- This week's discussion sections: local search and adversarial search
- HW3 due April 1
- Grades have been updated in Canvas
- Regrade requests + exam bonus by next week
- Ethics discussion #2: March 27

Today's Class

- Adversarial Search
  - Minimax algorithm
  - Alpha-beta pruning

Two-Player Games

- A special category of multiagent environment
  - Only one other agent (adversary)
  - Fully observable
  - Deterministic
- Characteristics of adversary
  - Objectives are exactly opposed to primary agent
  - Zero-sum game
- Agents move in alternating turns

Why Are These Problems Hard?

- Chess
  - Branching factor of 35
  - Games often include 50 moves per player
  - Search space: $35^{100} \sim 10^{154}$
  - Traditional search infeasible

- Robot soccer
  - Branching factor??
  - Consistent sensor input?
  - Balance?

When poll is active respond at **PollEv.com/492**

How to address challenges from two-player games? Select all valid.

How Will We Tackle This?

- New approaches
  - New definition for "optimal move"
  - Identify an algorithm to find the optimal move
- Pruning
  - Ignore less relevant portions of the search tree
  - Cut down on time and memory!
- Heuristic evaluation functions
  - Approximate true utility of a state without complete search (or with imperfect information)

Definition of a Game

- We will call our two players: MAX and MIN
  - MAX is the player that <u>moves first</u>
  - Take turns until the game is over

- At the end of the game:
  - Points awarded to winner
  - Penalties given to loser

Definition of a Game

- S_0: The initial state
- PLAYER(s): Defines which player has move in state
- ACTION(s): Returns set of legal moves in state
- RESULT(s,a): Transition model defines result of move

Definition of a Game

- TERMINAL-TEST(s):
  - Returns true when game is over (in a terminal state)
  - Returns false otherwise

- UTILITY(s,p): Utility function
  - Defines numeric value for a game that ends in terminal state, s, for player, p
  - Zero-sum game: total payoff to players is the same for every instance of the game

Game Tree

- Created using ACTIONS function and RESULT function
  - Vertices are states
  - Edges are moves

- So far, this looks extremely similar to search we've already looked at

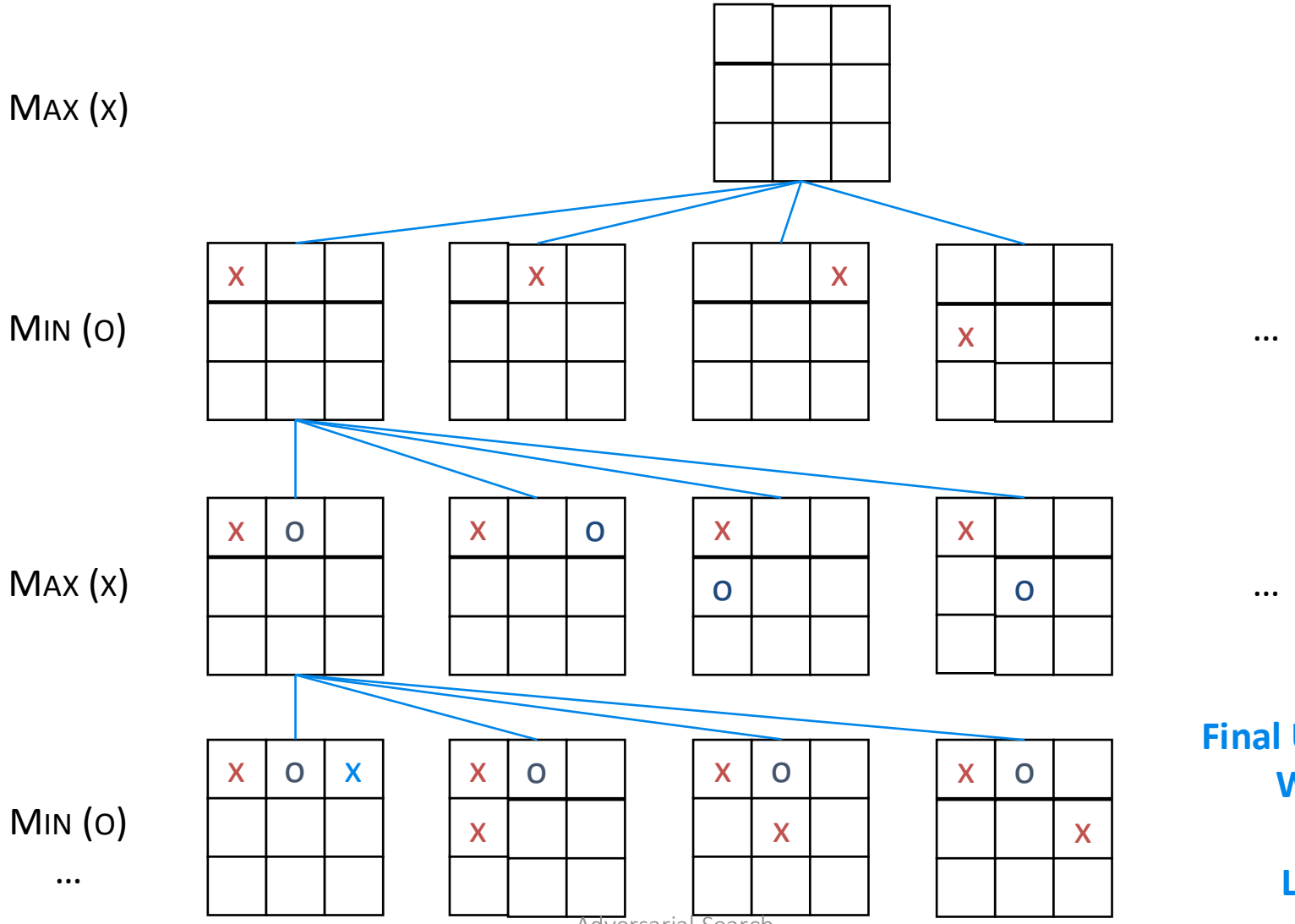
Simple Example: Tic-Tac-Toe

- Rules:
  - Take turns
  - First player: x
  - Second player: o
- Goal:
  - Get a row/column/diagonal of your three symbols before the other person

X	X	X
	O	
O		

Here: X wins!

Let’s Create a Game Tree – Tic-Tac-Toe



Final Utility:
Win: +1
Tie: 0
Loss: -1

Minimax Search

Optimal Decisions

- Normal search:
  - Set of actions leading to goal state – a win!
- Adversarial search:
  - MIN has an effect on the strategy
  - MAX must find a contingent strategy
- Contingent strategy:
  - Given initial state
  - Identify all moves given all responses by MIN
  - Then all MAX'S moves given MIN'S responses to those moves
  - Etc.

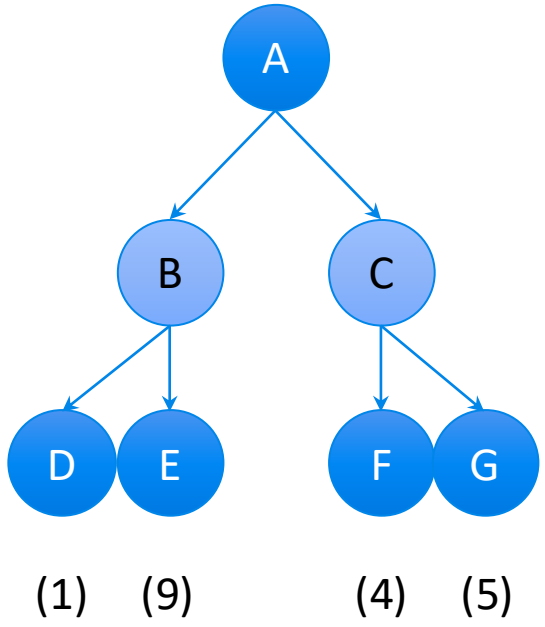
Optimal Decisions

Optimal strategy:

Leads to outcomes at least as good as any other strategy when one is playing an infallible opponent

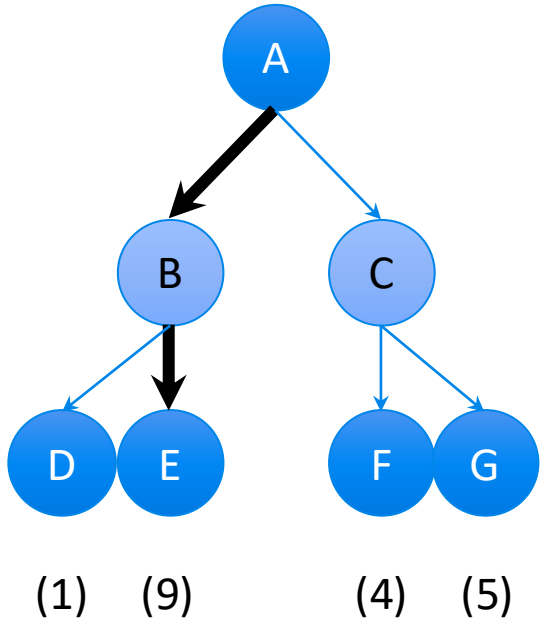
Defining Optimality

- Previously: cheapest sequence of actions leading to goal state
- Now: challenge because Min tries to win!



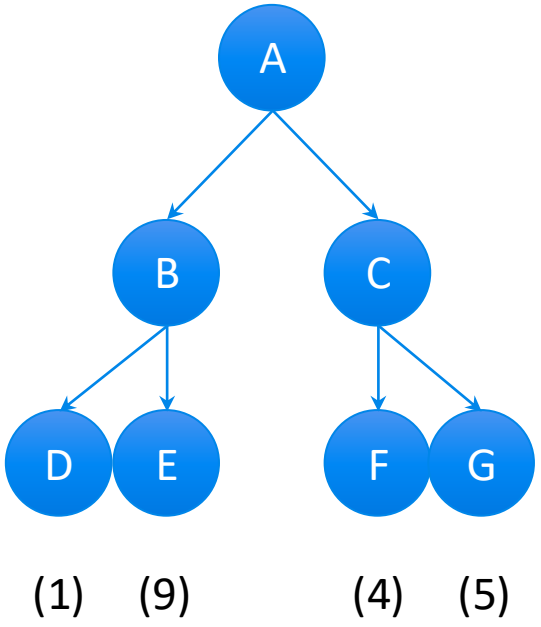
Defining Optimality

- Previously: cheapest sequence of actions leading to goal state
- Now: challenge because Min tries to win!



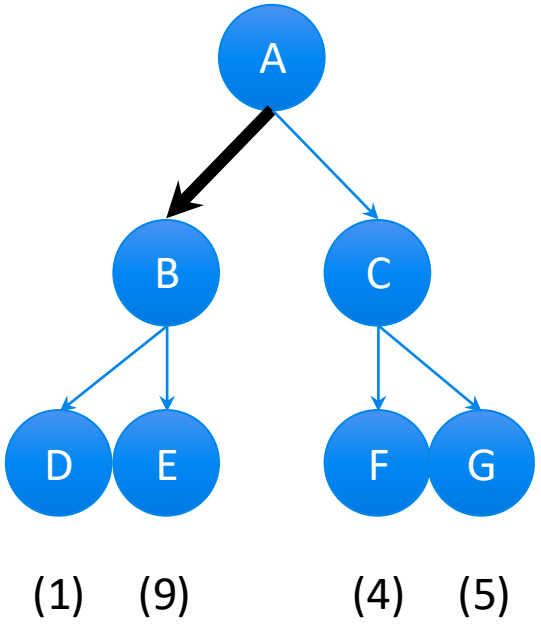
Defining Optimality

- Previously: cheapest sequence of actions leading to goal state
- Now: challenge because Min tries to win!



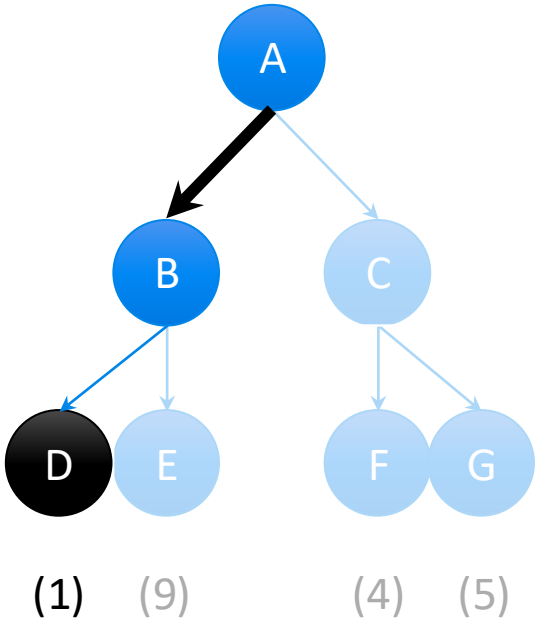
Defining Optimality

- Previously: cheapest sequence of actions leading to goal state
- Now: challenge because Min tries to win!



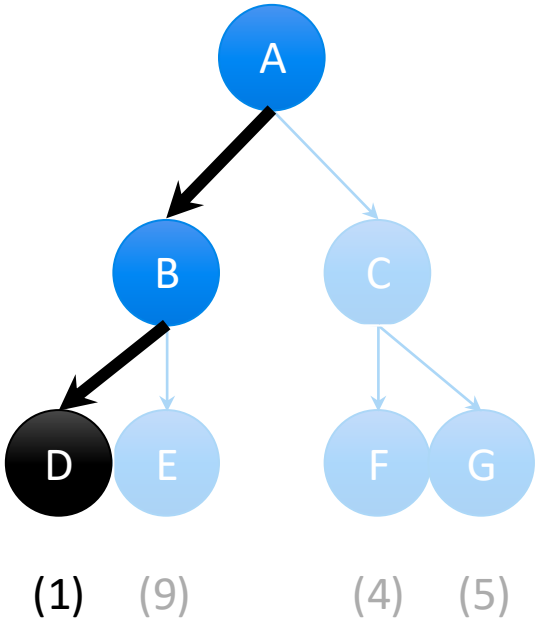
Defining Optimality

- Previously: cheapest sequence of actions leading to goal state
- Now: challenge because Min tries to win!



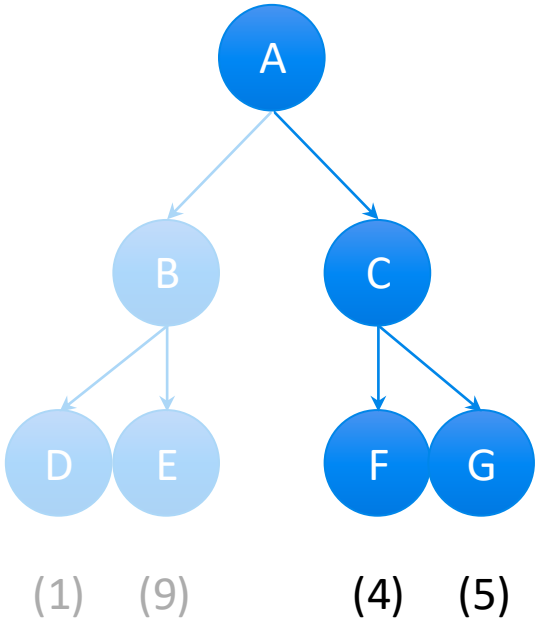
Defining Optimality

- Previously: cheapest sequence of actions leading to goal state
- Now: challenge because Min tries to win!



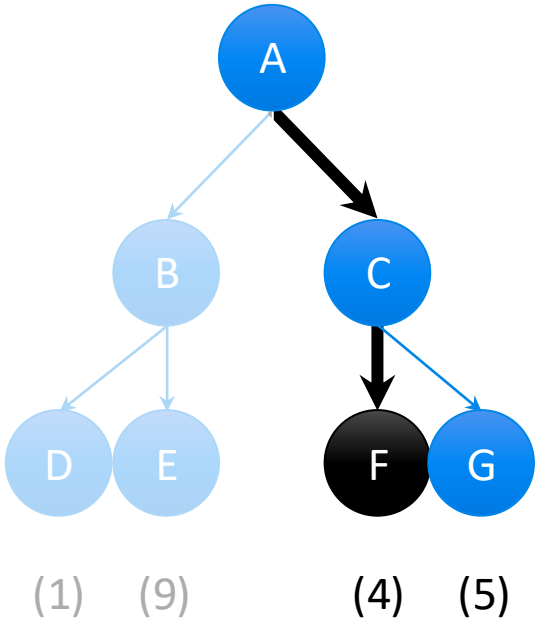
Defining Optimality

- Previously: cheapest sequence of actions leading to goal state
- Now: challenge because Min tries to win!



Defining Optimality

- Previously: cheapest sequence of actions leading to goal state
- Now: challenge because Min tries to win!



Optimal Strategy – Minimax!

- Need a **contingent** strategy:
  - Calculate the value of each node – MINIMAX(n)
  - Calculates utility (for MAX) of being in given state
  - Assumes that both players play optimally!
  - Minimax value of terminal node – its utility

$$\text{MINIMAX}(s) = \begin{cases} \text{UTILITY}(s) & \text{if TERMINAL-TEST}(s) \\ \max_{a \in Actions(s)} \text{MINIMAX}(\text{RESULT}(s,a)) & \text{if PLAYER}(s) = \text{MAX} \\ \min_{a \in Actions(s)} \text{MINIMAX}(\text{RESULT}(s,a)) & \text{if PLAYER}(s) = \text{MIN} \end{cases}$$

Generally:
MAX prefers state with maximum value
MIN prefers state with minimum value

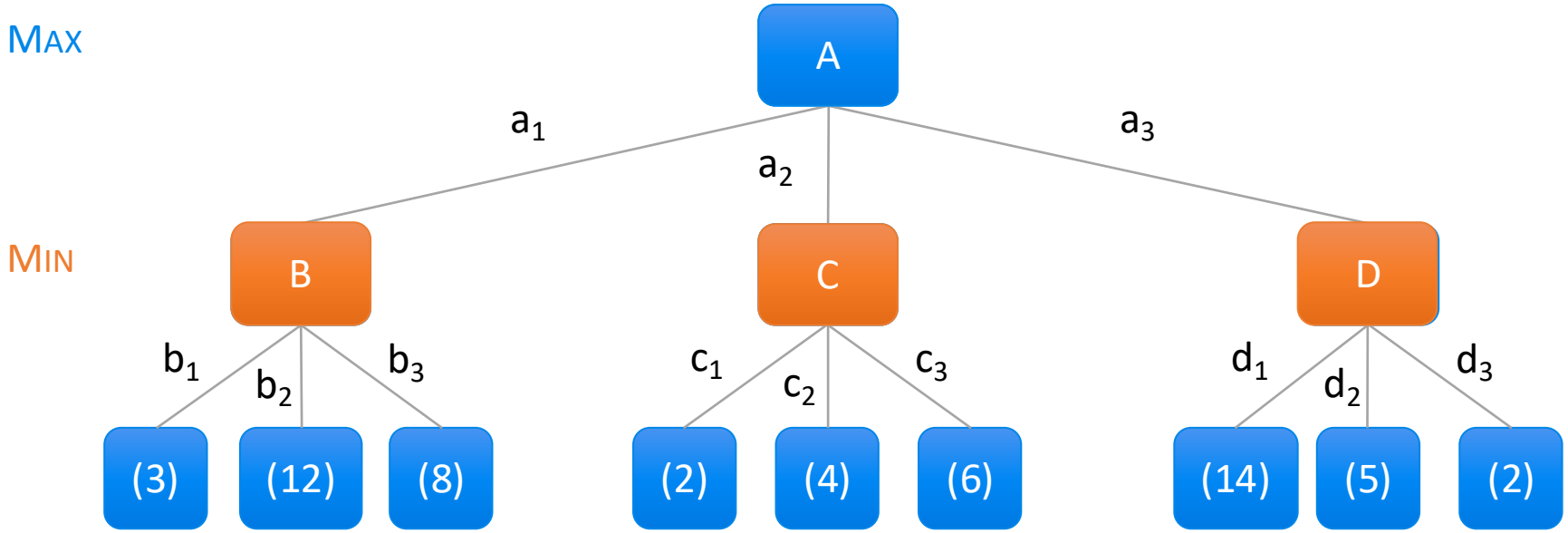
Simpler Example

- Tic-Tac-Toe: too complicated to fit on one slide

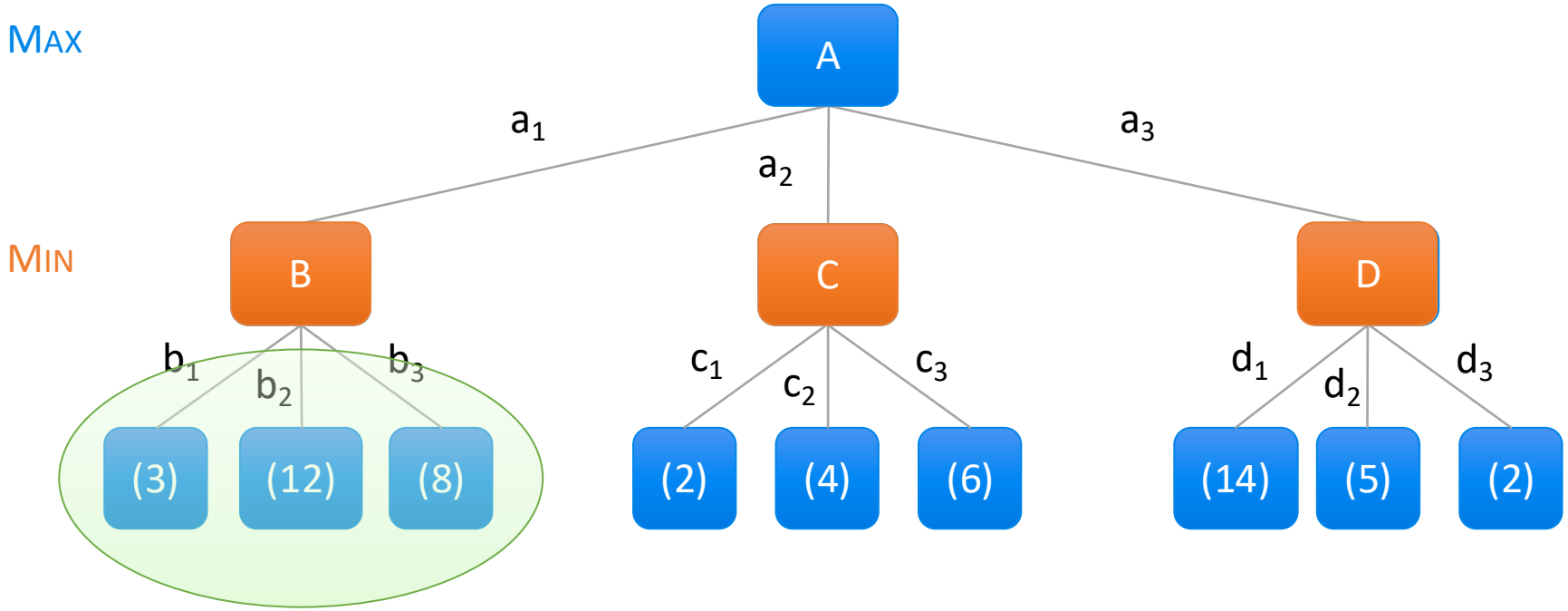
- Simpler game: A-B
  - Each side (MAX, MIN) has one turn
  - Possible moves for Max at root node: a_1, a_2, a_3
  - Possible moves for Min at following node: b_1, b_2, b_3
  - Etc.

- Utilities of state: 2 – 14

Work Through Example



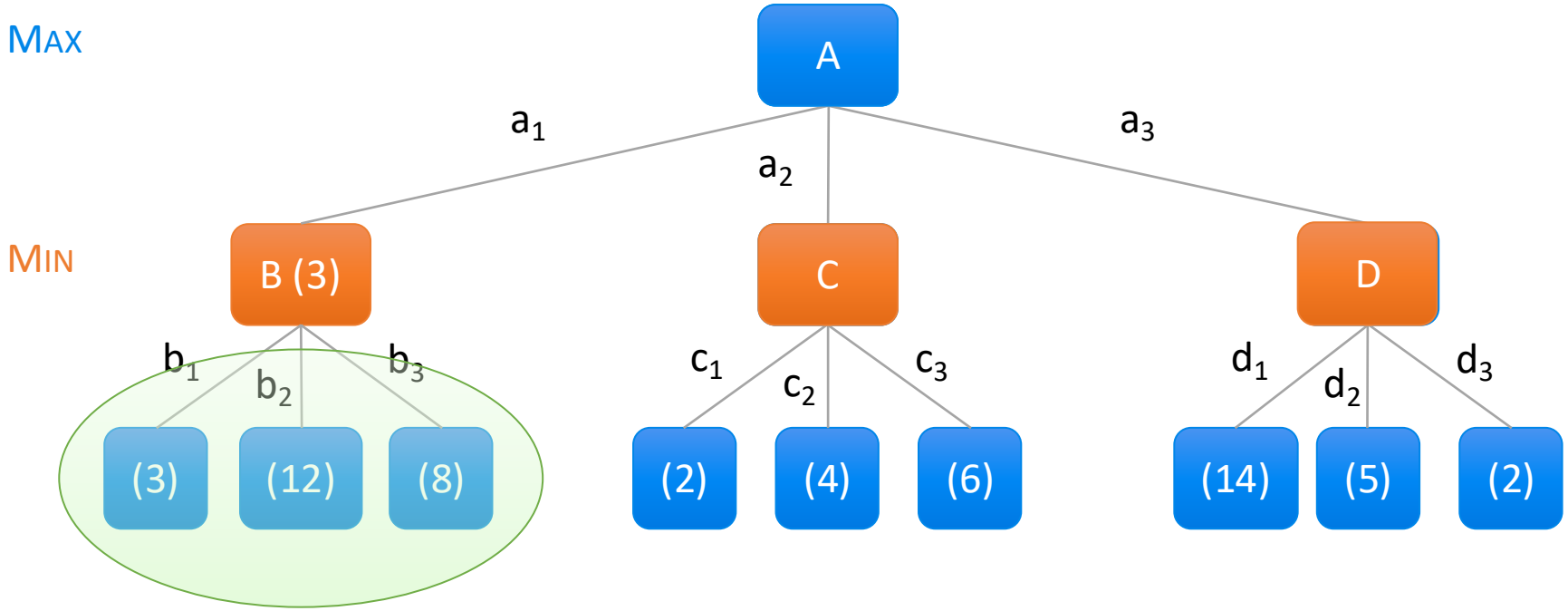
Work Through Example



Let's apply Minimax:

Utility (Min): min(3, 12, 8) = 3

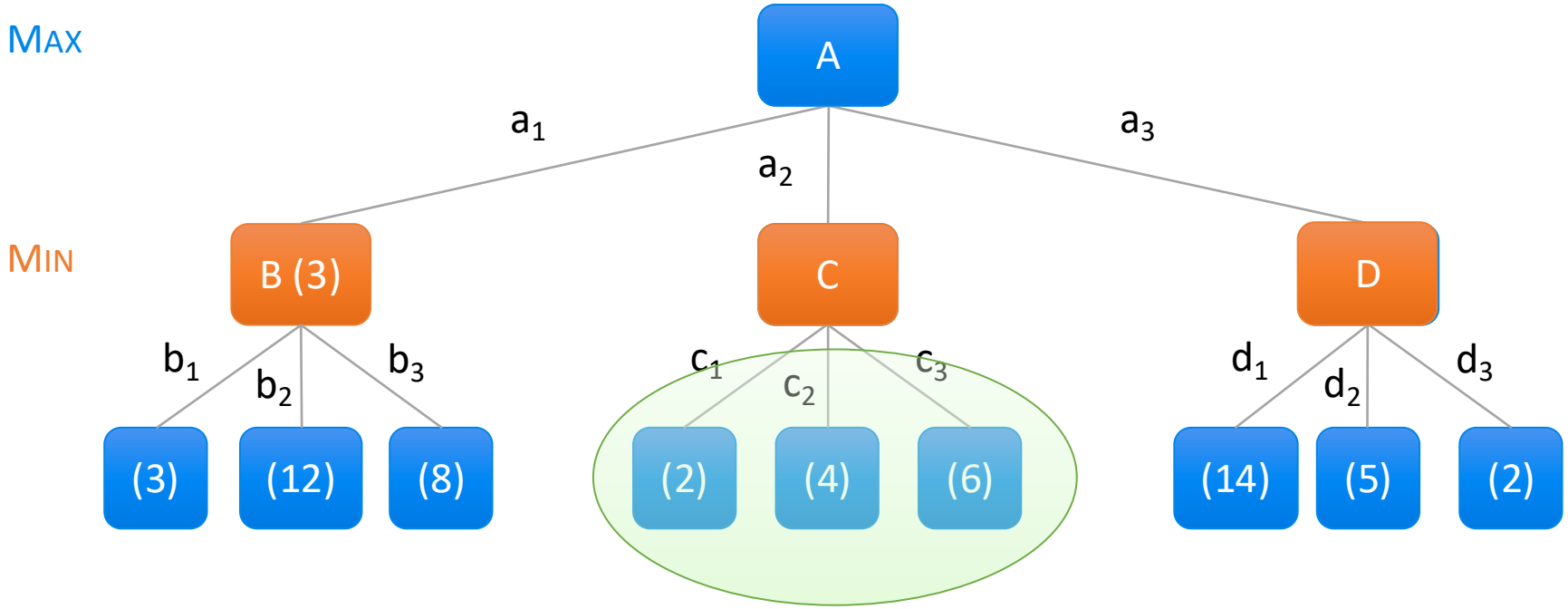
Work Through Example



Let's apply Minimax:

Utility (Min): min(3, 12, 8) = 3

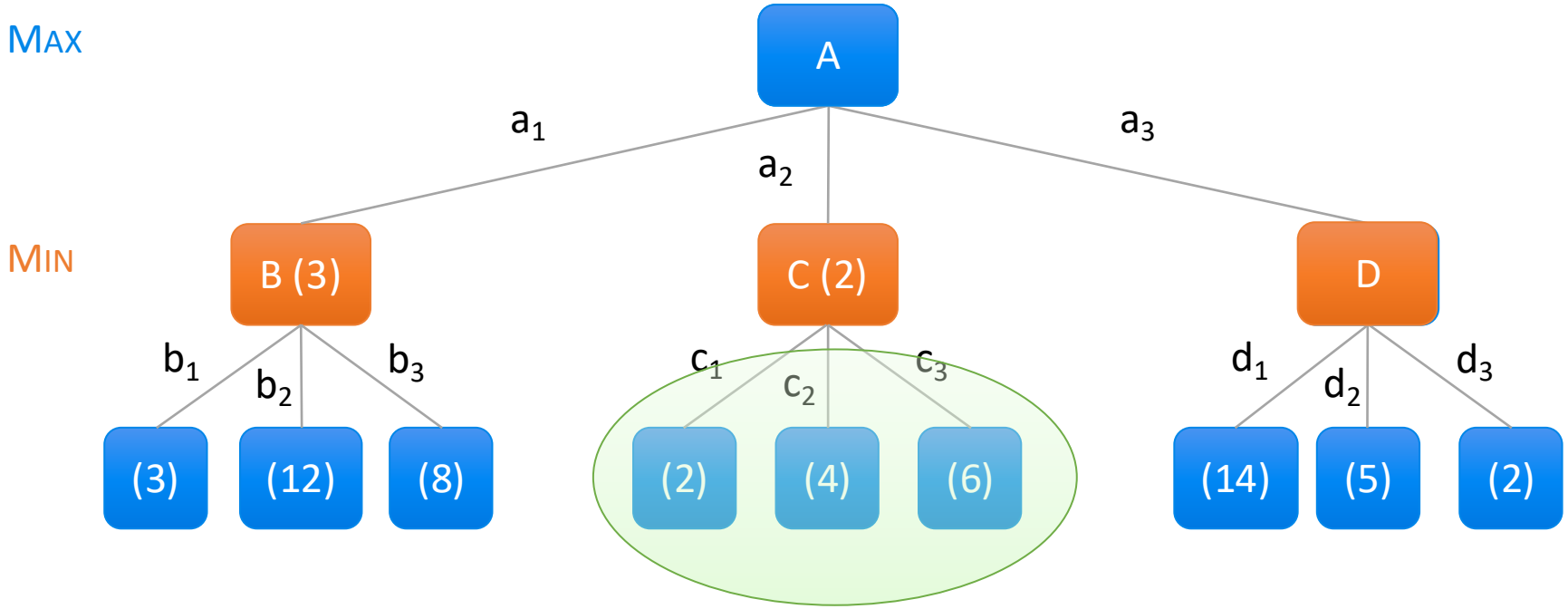
Work Through Example



Let's apply Minimax:

Utility (Min): min(2, 4, 6) = 2

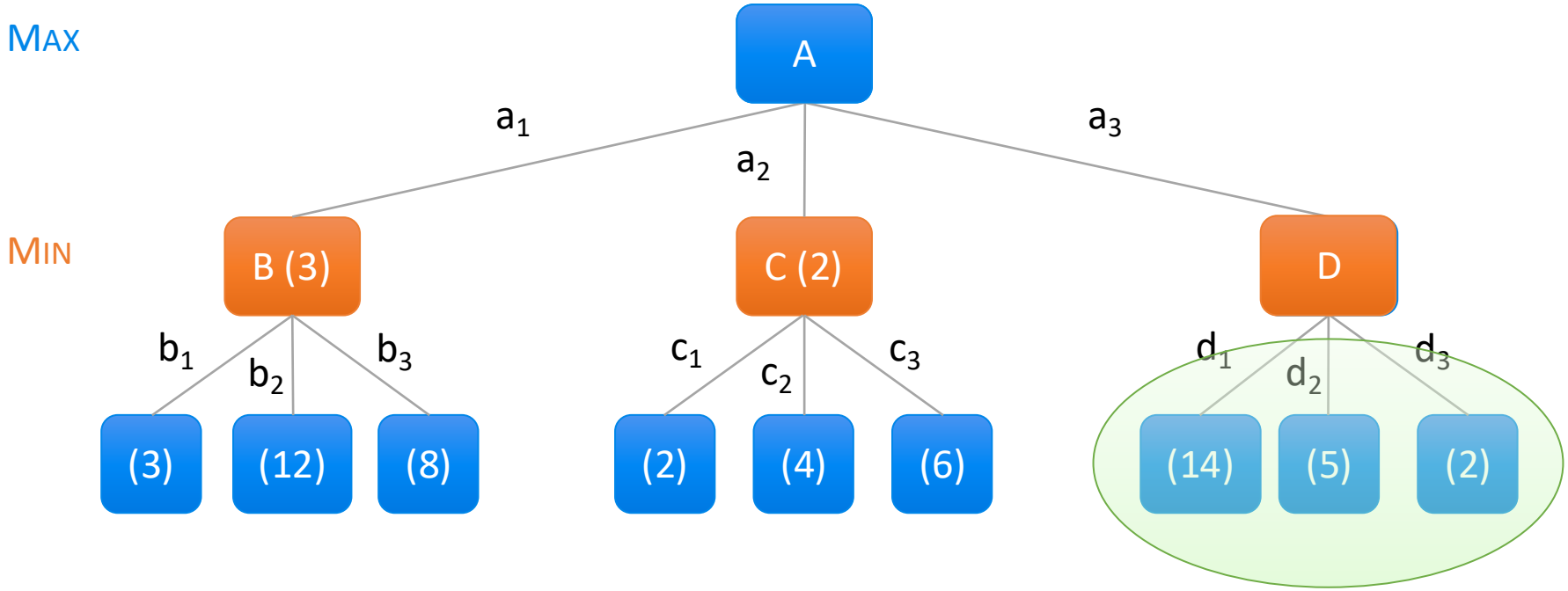
Work Through Example



Let's apply Minimax:

Utility (Min): min(2, 4, 6) = 2

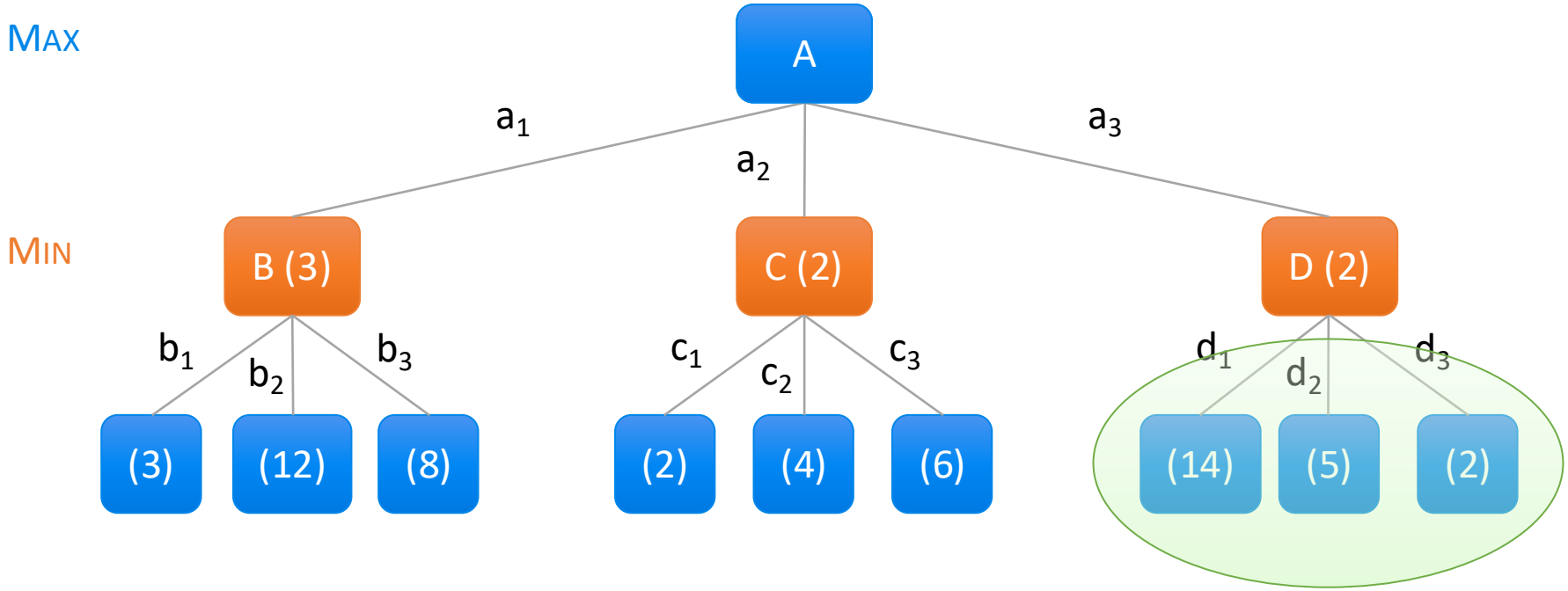
Work Through Example



Let's apply Minimax:

Utility (Min): min(14, 5, 2) = 2

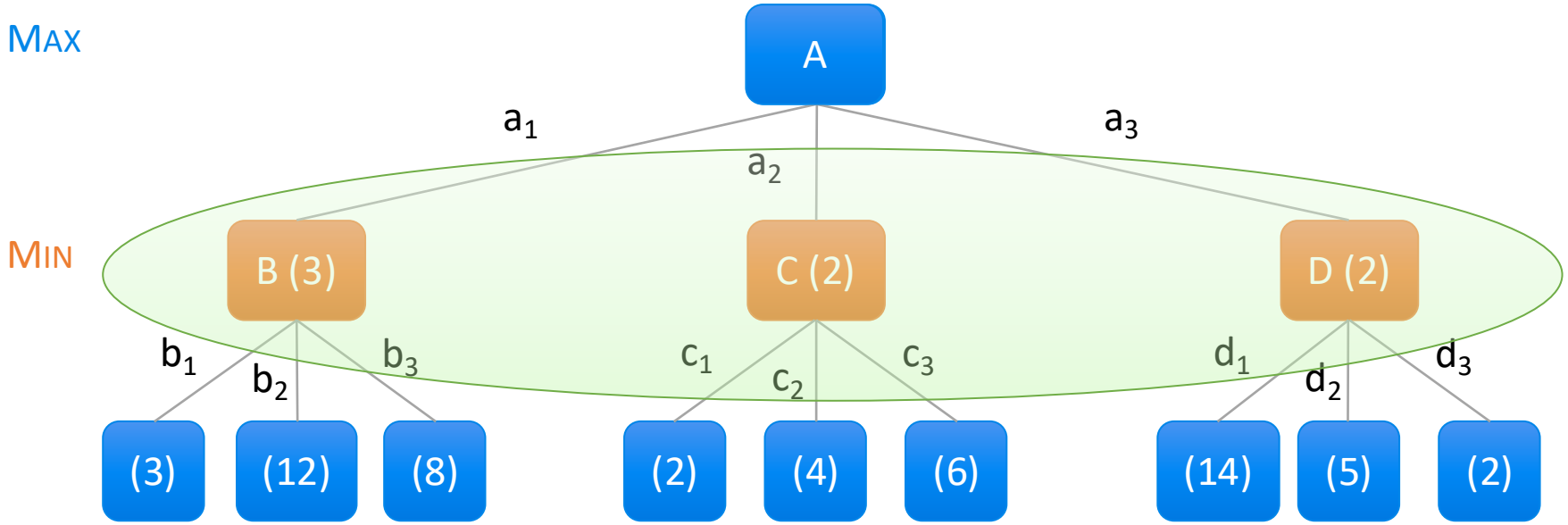
Work Through Example



Let's apply Minimax:

Utility (Min): min(14, 5, 2) = 2

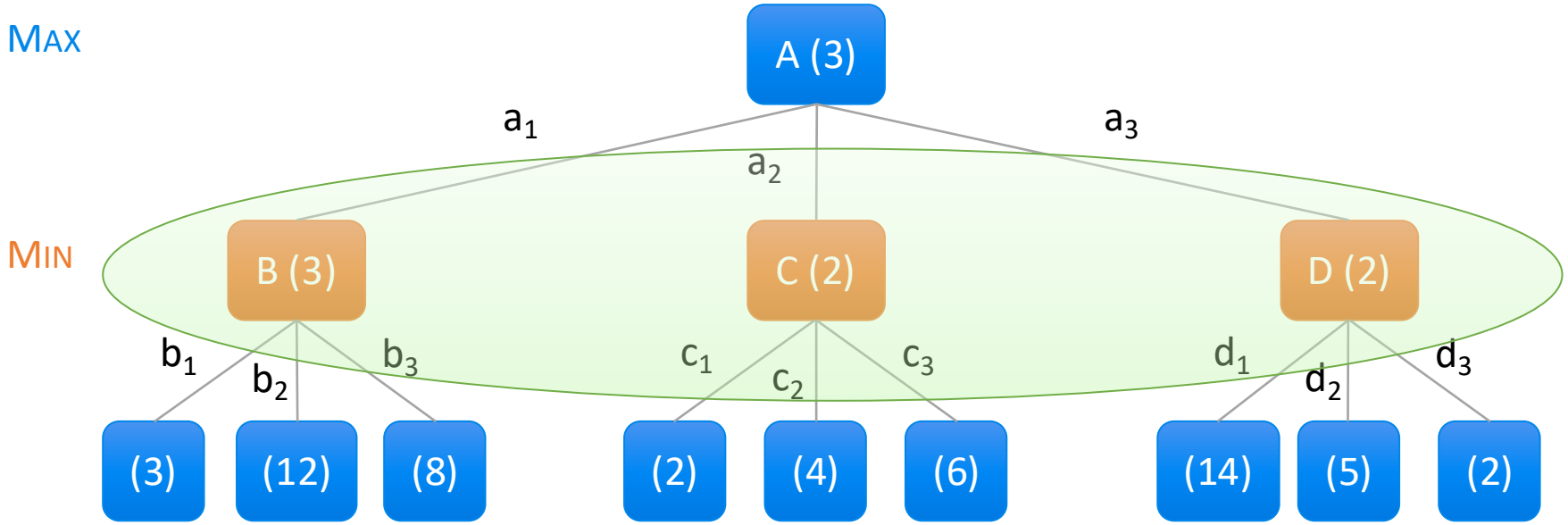
Work Through Example



Let's apply Minimax:

Utility (Max): max(3, 2, 2) = 3

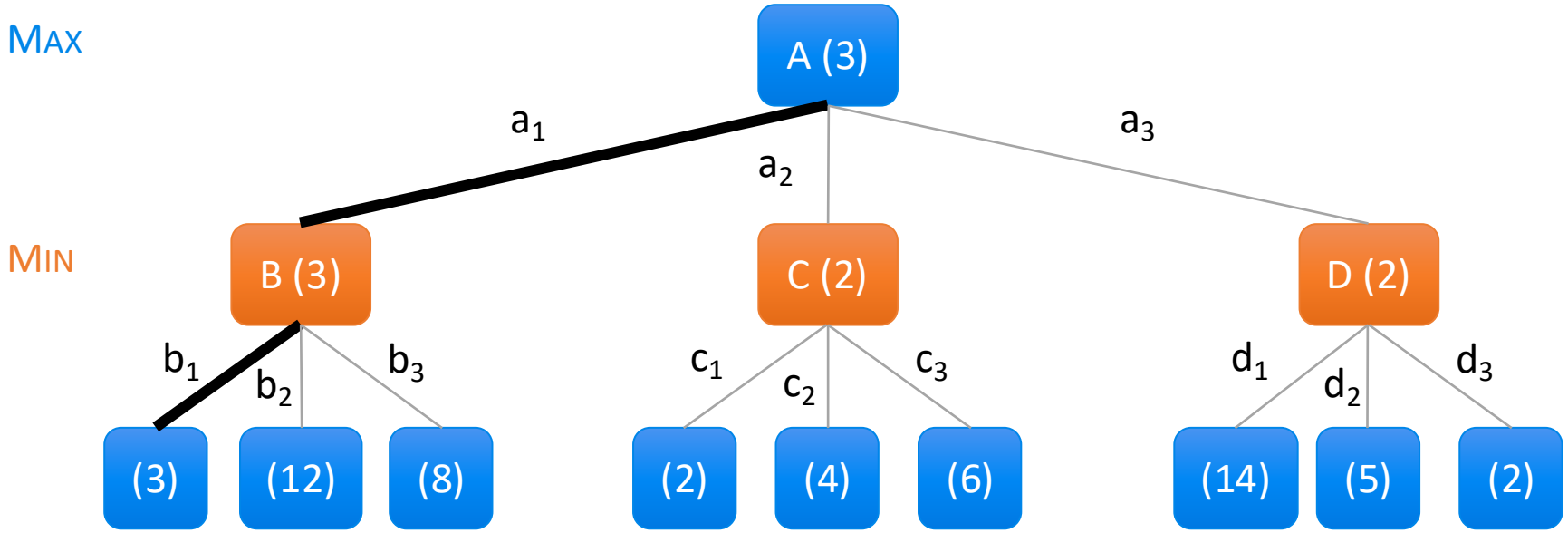
Work Through Example



Let's apply Minimax:

Utility (Max): max(3, 2, 2) = 3

Work Through Example



Minimax Implementation

```
def max-value(state):
    initialize v = -∞
    for each successor of state:
        v = max(v, min-value(successor))
    return v
```

$$V(s) = \max_{s' \in \text{successors}(s)} V(s')$$

```
def min-value(state):
    initialize v = +∞
    for each successor of state:
        v = min(v, max-value(successor))
    return v
```

$$V(s') = \min_{s \in \text{successors}(s')} V(s)$$

Minimax Implementation (Alternative)

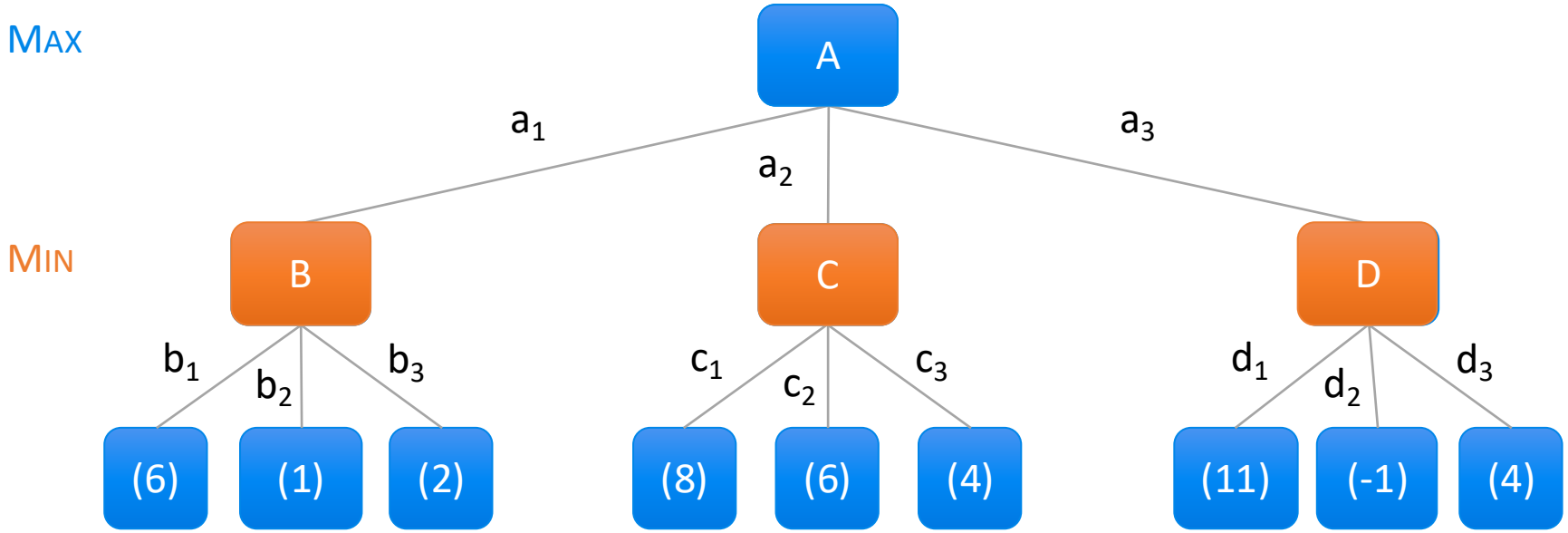
```
def value(state):
    if the state is a terminal state: return the state's utility
    if the next agent is MAX: return max-value(state)
    if the next agent is MIN: return min-value(state)
```

```
def max-value(state):
    initialize v = -∞
    for each successor of state:
        v = max(v, value(successor))
    return v
```

```
def min-value(state):
    initialize v = +∞
    for each successor of state:
        v = min(v, value(successor))
    return v
```

Your turn!

When poll is active respond at **PollEv.com/492**



Properties

- How efficient is minimax?
  - Maximal depth: m
  - Legal moves at each point: b

- Time complexity: $O(b^m)$ ← This is not good
- Space complexity:
  - O(bm) – generate all actions at once
  - O(m) – generate one action at a time

Limitations

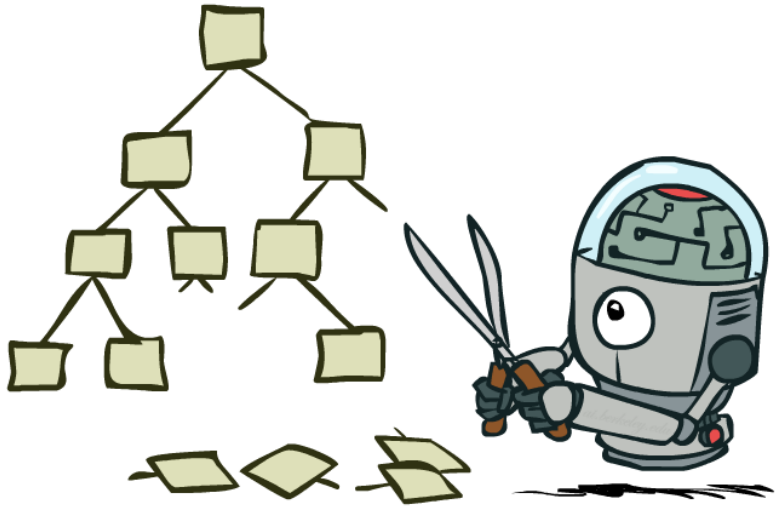
- Chess
  - Branching factor of 35
  - Games often include 50 moves/player
  - Search space: $35^{100} \sim 10^{154}$
  - Traditional search infeasible

- This is still a problem!
  - Minimax has to explore the entire tree!

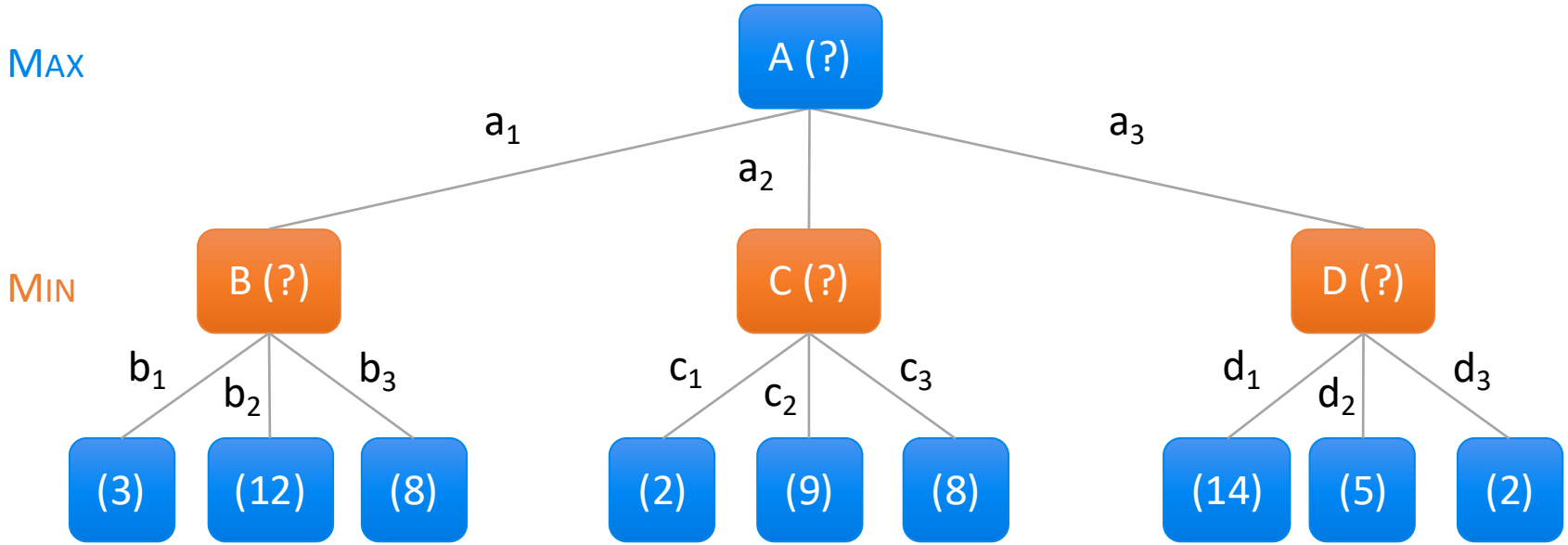
Alpha-Beta Pruning

What Can We Do?

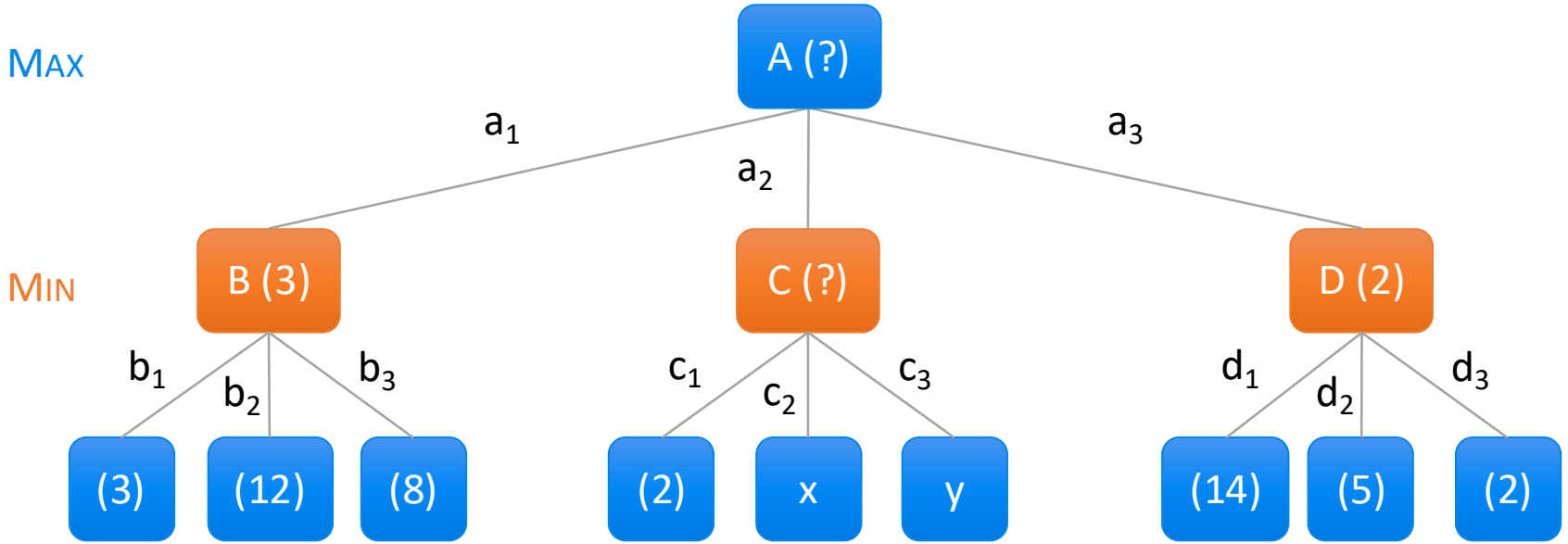
- What are we doing?
  - Assuming that MIN always has our worst interest at heart
- What if we ignore moves that MIN would ignore?
  - Prune the search tree!
  - Alpha-Beta Pruning



Alpha-Beta Pruning – Intuition



Alpha-Beta Pruning – Intuition



MINIMAX(root) = max(min(3, 12, 8), min(2, x, y), min(14, 5, 2))

= max(3, min(2, x, y), 2))

= max(3, z, 2) where z = min(2, x, y) $\leq$ 2

= 3

Decision independent of values of pruned leaves x and y!

Remember Minimax Implementation (Alternative)

```
def value(state):
    if the state is a terminal state: return the state's utility
    if the next agent is MAX: return max-value(state)
    if the next agent is MIN: return min-value(state)
```

```
def max-value(state):
    initialize v = -∞
    for each successor of state:
        v = max(v, value(successor))
    return v
```

```
def min-value(state):
    initialize v = +∞
    for each successor of state:
        v = min(v, value(successor))
    return v
```

Alpha-Beta Implementation

α: MAX's best option on path to root
β: MIN's best option on path to root

Think: α = "at least"
Think: β = "at most"

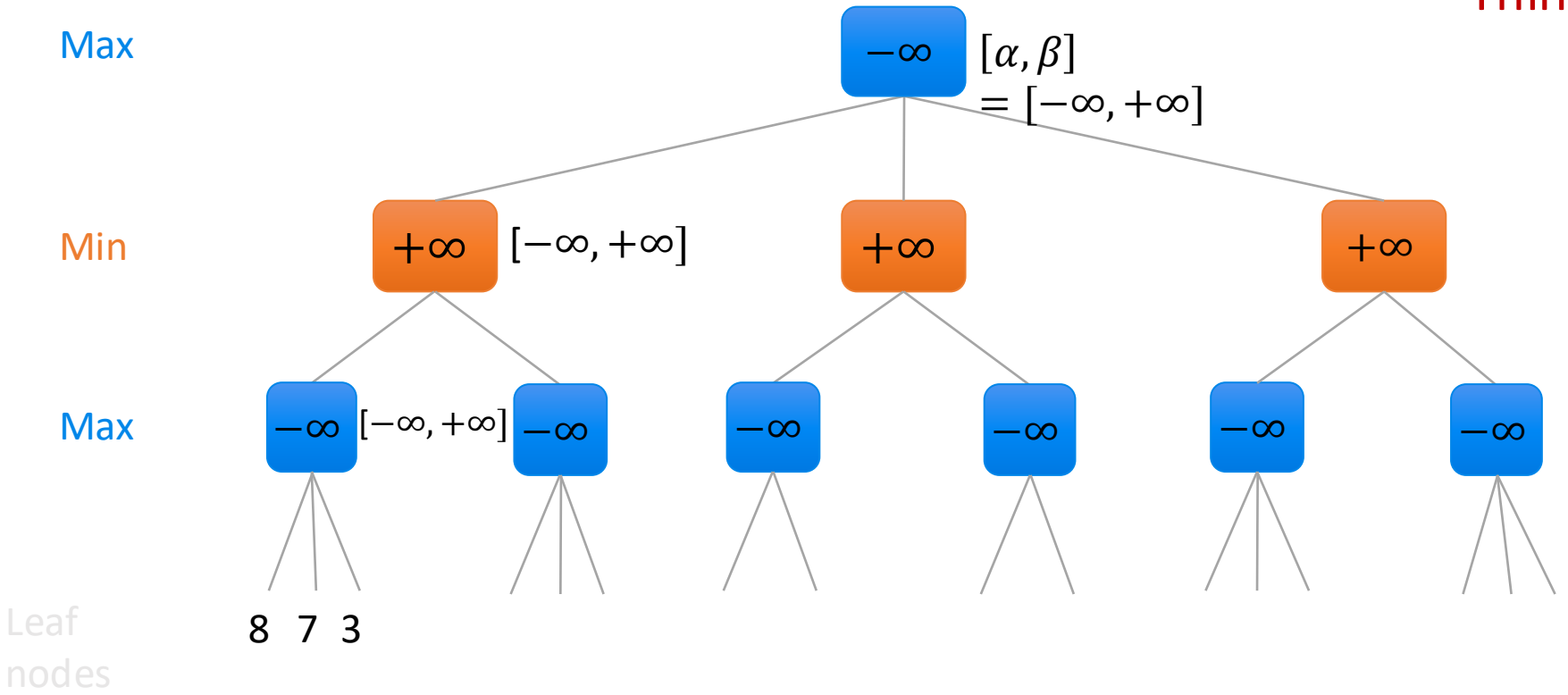
```
def max-value(state, α, β):
    initialize v = -∞
    for each successor of state:
        v = max(v, value(successor, α, β))
        if v ≥ β return v
        α = max(α, v)
    return v
```

```
def min-value(state, α, β):
    initialize v = +∞
    for each successor of state:
        v = min(v, value(successor, α, β))
        if v ≤ α return v
        β = min(β, v)
    return v
```

Alpha-Beta Pruning

α: MAX's best option on path to root
β: MIN's best option on path to root

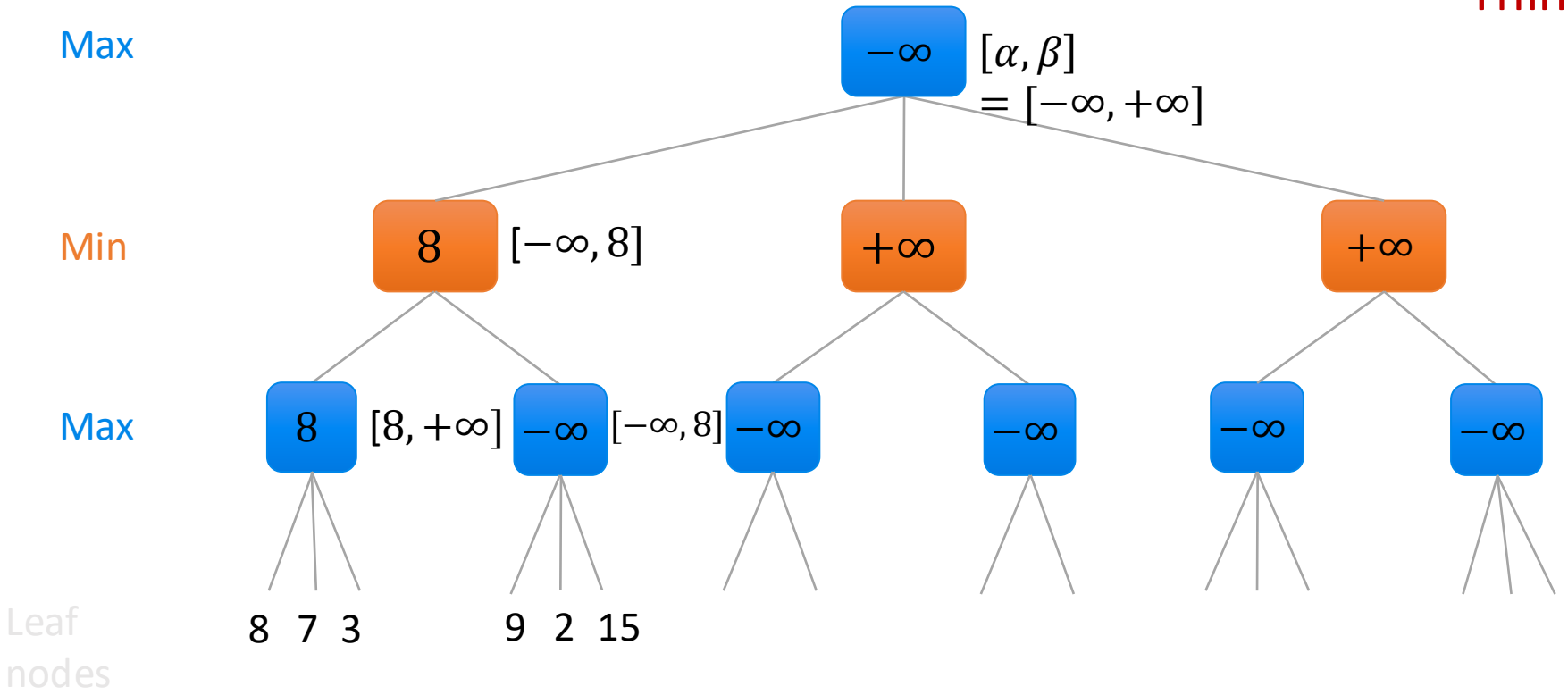
Think: α = "at least"
Think: β = "at most"



Alpha-Beta Pruning

α: MAX's best option on path to root
β: MIN's best option on path to root

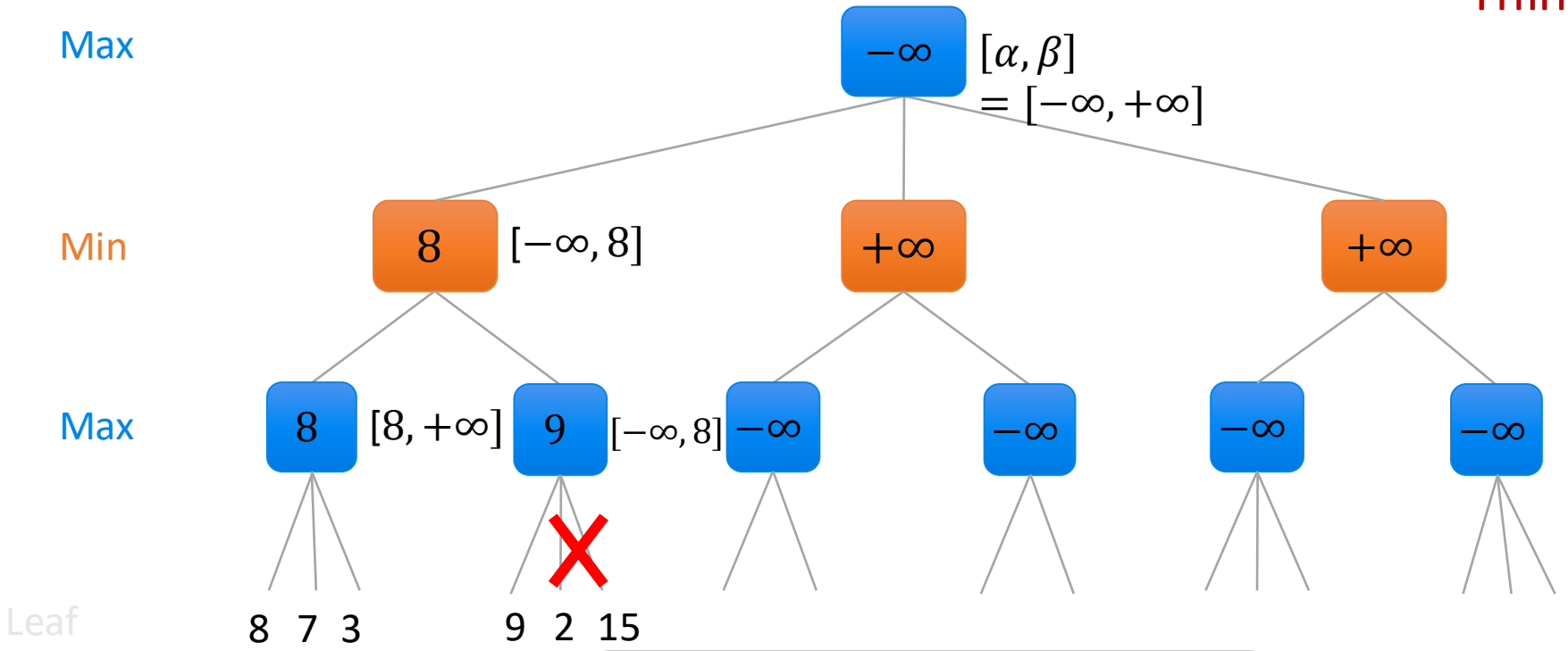
Think: α = "at least"
Think: β = "at most"



Alpha-Beta Pruning

α: MAX's best option on path to root
β: MIN's best option on path to root

Think: α = "at least"
Think: β = "at most"

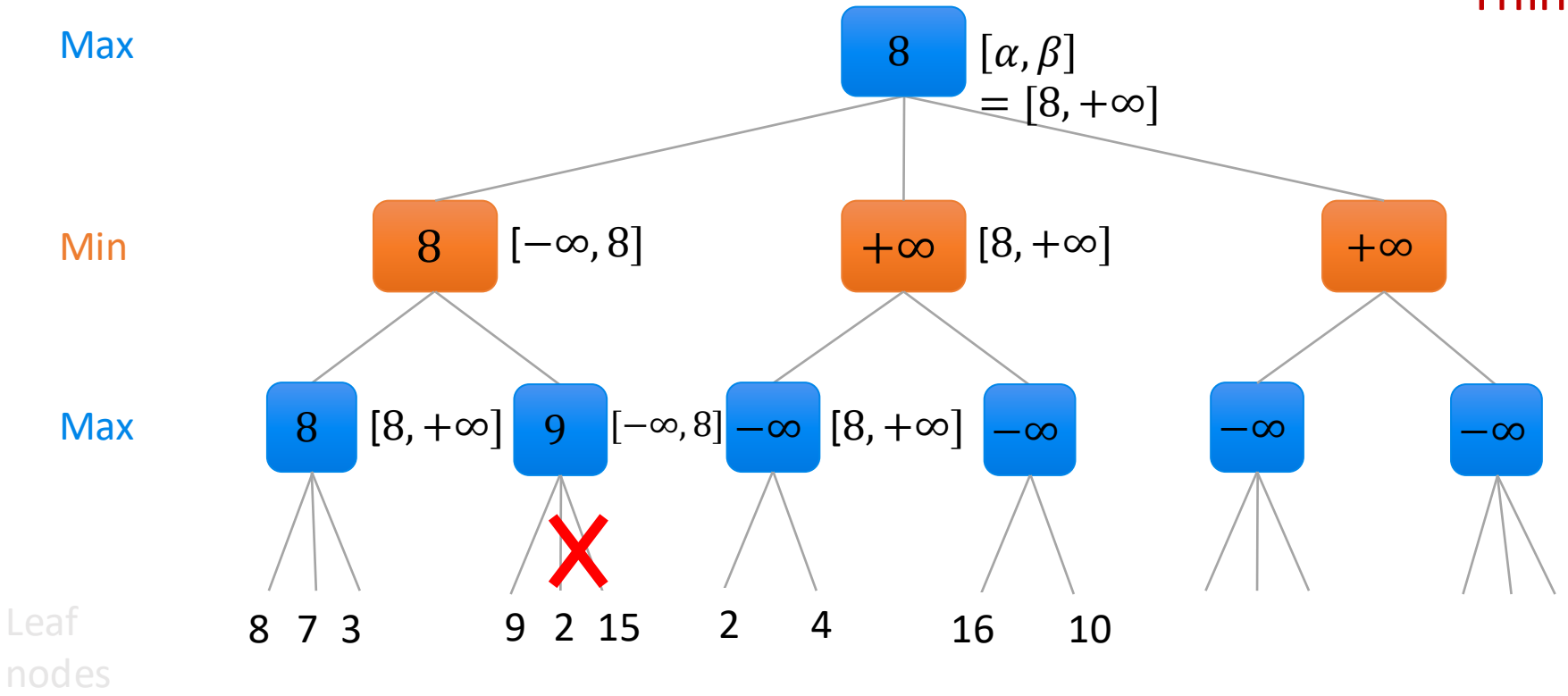


Prune: Value of node $v = 9$ is higher than MIN's best option $\beta = 8$

Alpha-Beta Pruning

α: MAX's best option on path to root
β: MIN's best option on path to root

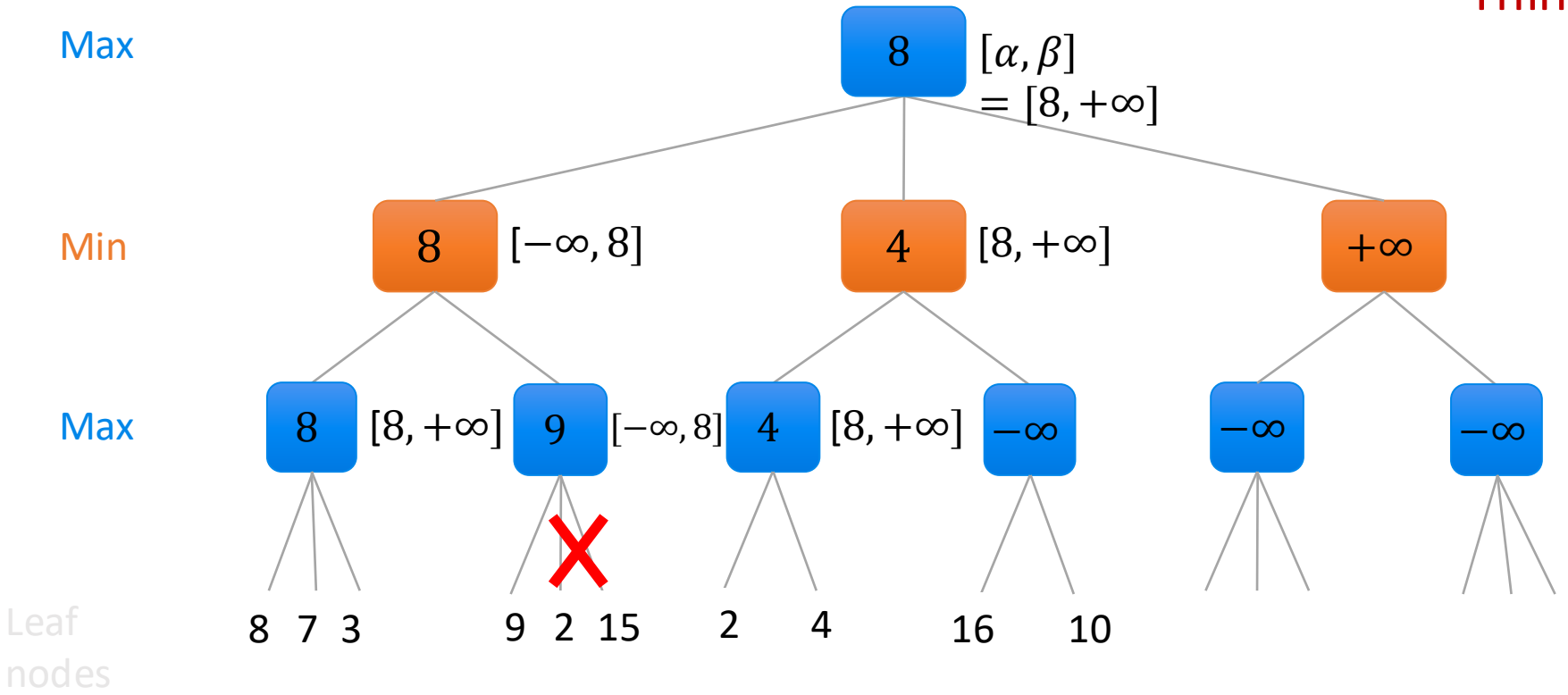
Think: α = "at least"
Think: β = "at most"



Alpha-Beta Pruning

α: MAX's best option on path to root
β: MIN's best option on path to root

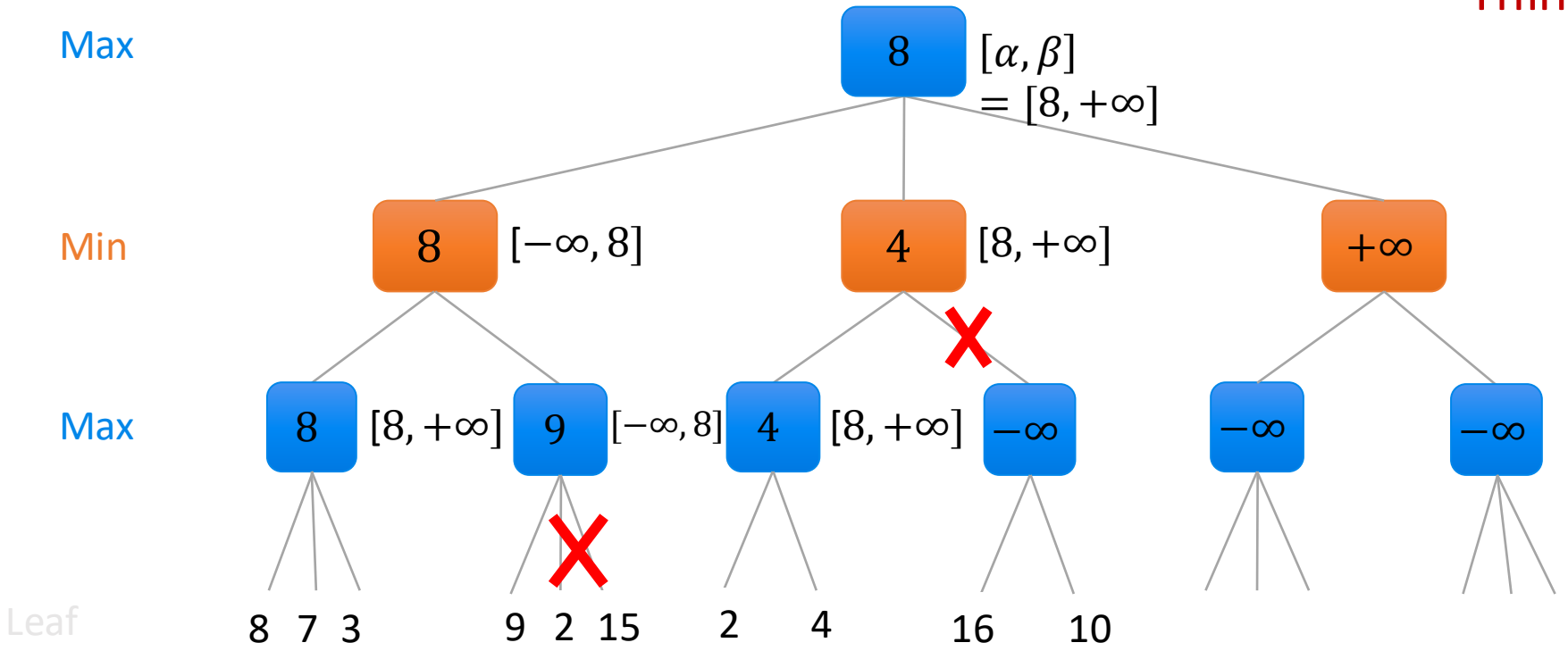
Think: α = "at least"
Think: β = "at most"



Alpha-Beta Pruning

α: MAX's best option on path to root
β: MIN's best option on path to root

Think: α = "at least"
Think: β = "at most"

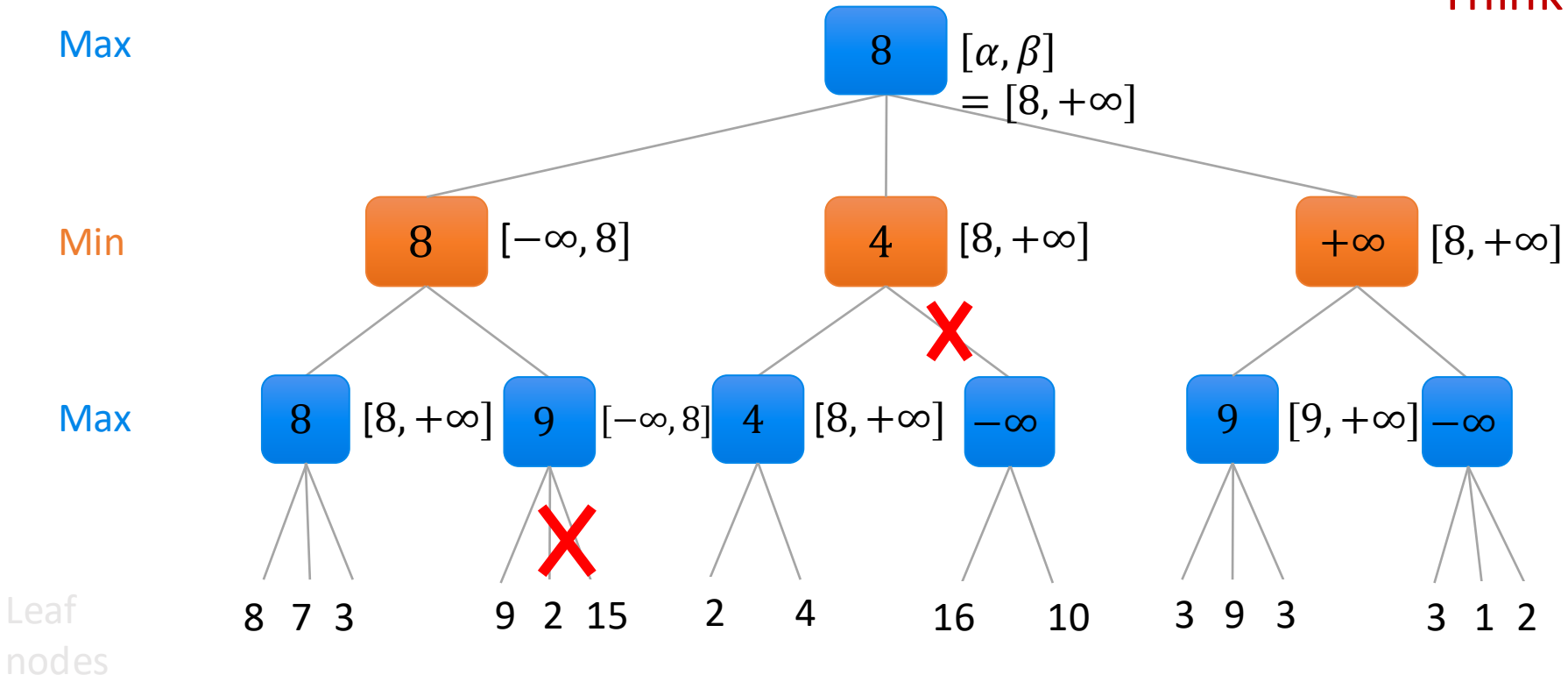


Prune: Value of node $v = 4$ is lower than MAX's best option $\alpha = 8$

Alpha-Beta Pruning

α: MAX's best option on path to root
β: MIN's best option on path to root

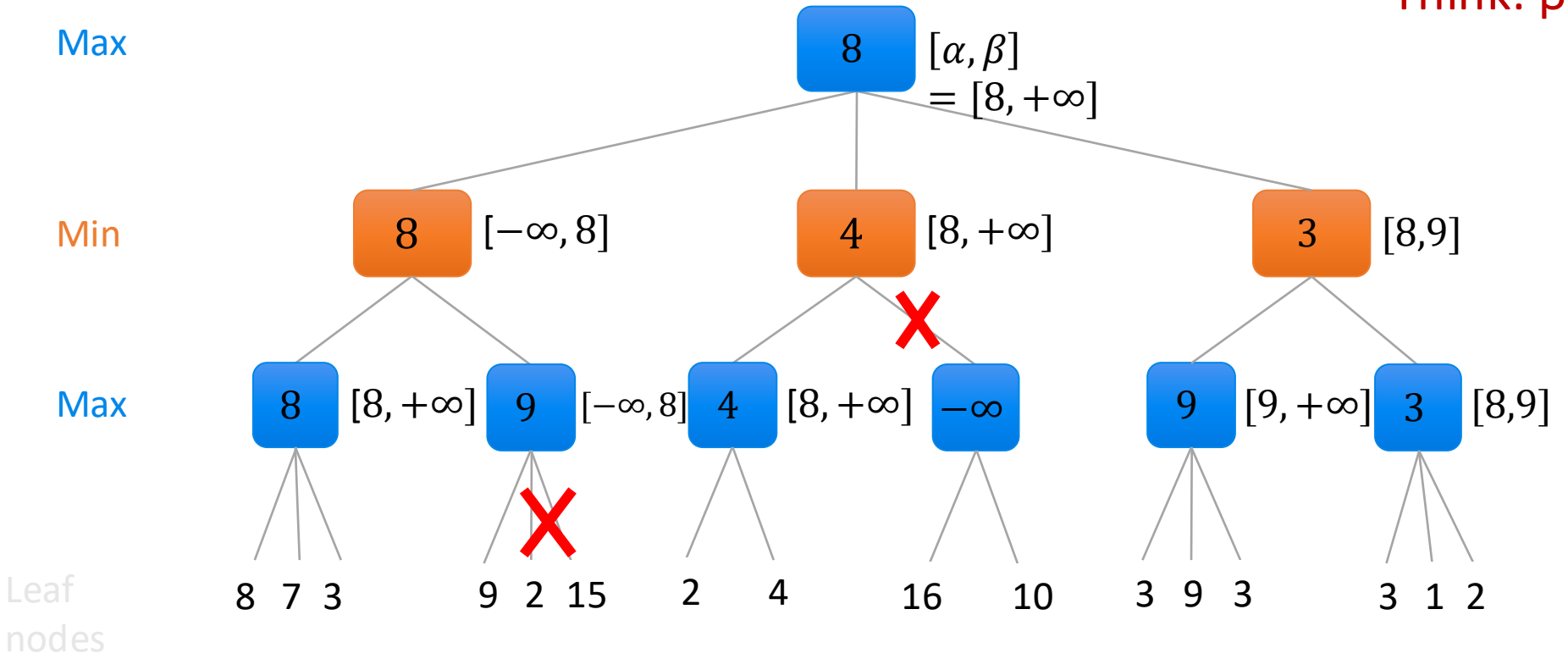
Think: α = "at least"
Think: β = "at most"



Alpha-Beta Pruning (Final Values)

α: MAX's best option on path to root
β: MIN's best option on path to root

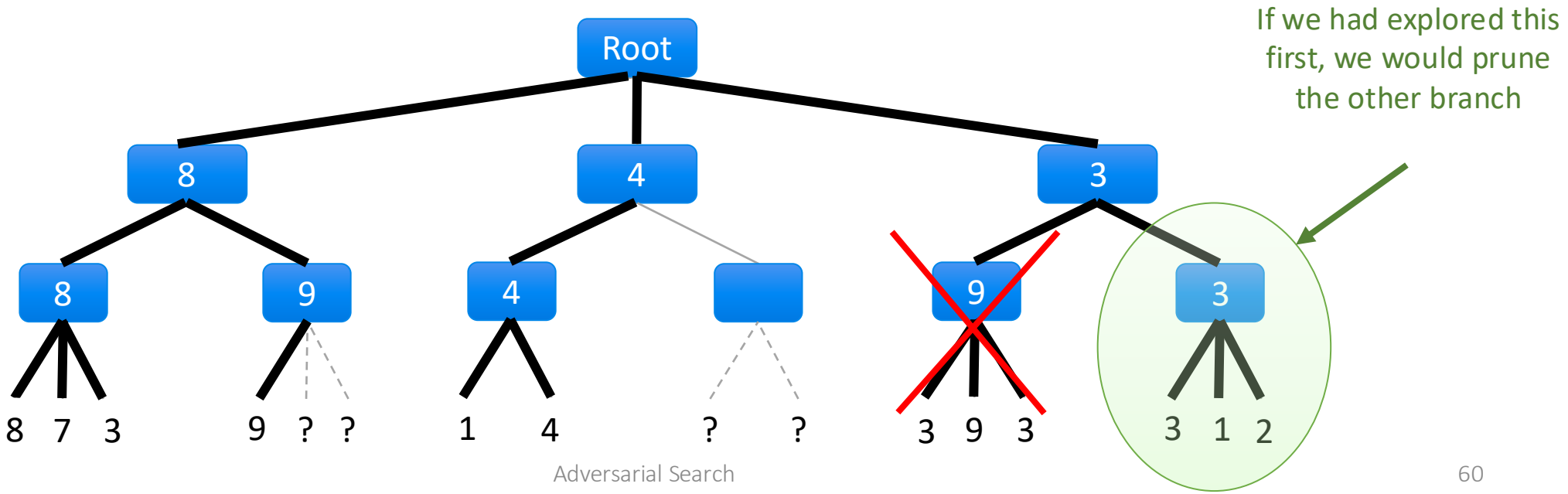
Think: α = "at least"
Think: β = "at most"



Alpha-Beta Pruning Effectiveness

- What kind of search is this?
  - Depth-first search!

- Highly dependent on the ordering of states



Alpha-Beta Overview

- Can be applied to trees of any depth
- Often can prune entire subtrees!

- Consider node n somewhere in the tree
  - If player has better choice m at a parent node of n then n will never be reached during play
  - Once we've learned enough about n (via descendants) we can prune it

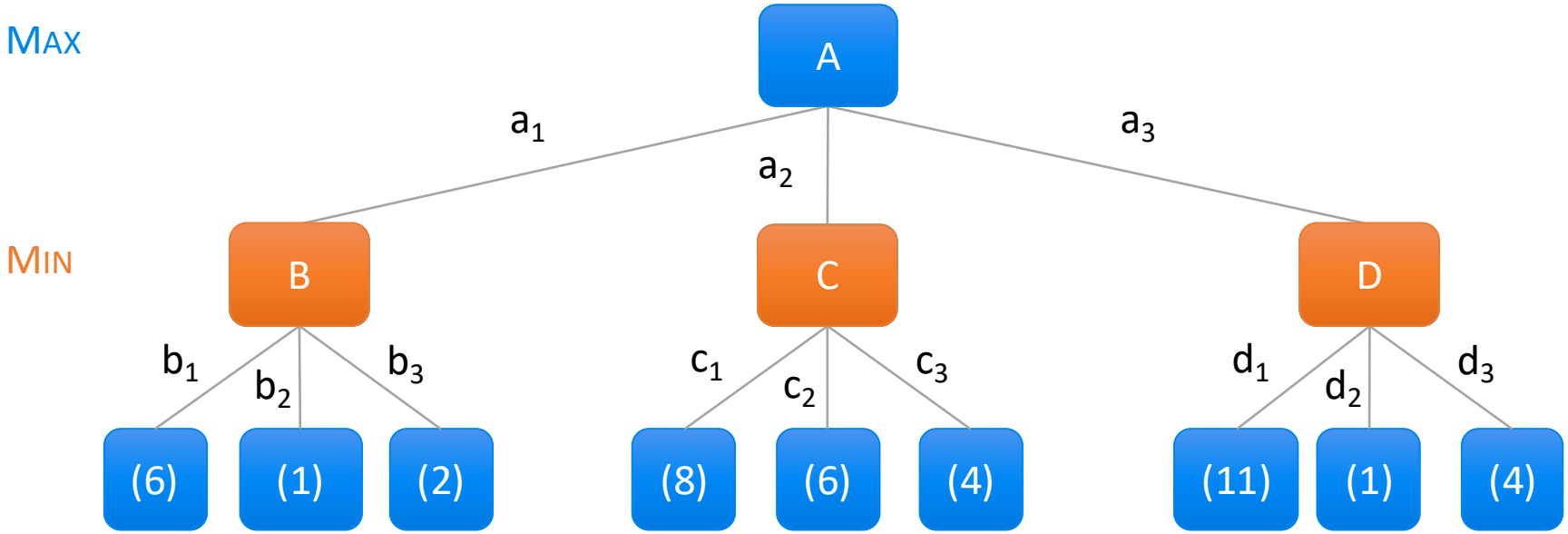
- Definitions:
  - α = the value of the best choice so far for MAX
  - β = the value of the best choice so far for MIN

Your turn!

Recall:
In each non-leaf node, we keep track of v, [α, β]
When
MAX: check v ≥ β; set α
MIN: check v ≤ α; set β

PollEv.com/492

- Assume we looking at the nodes from left to right. What nodes can we prune?



Time Complexity of Alpha-Beta

- If we can identify the best successors first:
  - Need to examine $O(b^{m/2})$ nodes! (ideal case)
  - Random case – $O(b^{3m/4})$
  - Worst case – $O(b^m)$

- What does this mean for the branching factor?
  - In the ideal case: $b^* = \sqrt{b}$
  - For chess – branching decreases to 6 (from 35)

Running in “Real-time” (How can we improve the efficiency of adversarial search?)

Issue:

We still have exponents in time!

Evaluation Functions

- What if we knew how good a state would ultimately turn out to be?
  - We could still do alpha-beta pruning!

- Properties:
  - Should order terminal states in same way as utility function!
  - Computation must not take too long
  - Should be strongly correlated with actual chances of winning for non-terminal states

Evaluation Functions

- If we stop searching before a terminal state, the algorithm will be *uncertain* about the final outcome of those states
  - Given infinite computation, we could calculate the exact utility of the state
  - Information limitation vs. computation limitation!

Evaluation Function Example

- Let's pretend that chess stems on # of pawns in endgame
  - One pawn
  - Two pawns

- For two pawns vs. one pawn
  - 72% of the time – victory!
  - 20% of the time – loss
  - 8% of the time – draw

Why is this not a straight-forward calculation?

- Expected value = (0.72*1) + (0.2*0) + (0.08*0.5) = 0.76

Evaluation Function via Features

- Let's use properties of the environment
  - Features – describe current world state

$$\textsc{Eval}(s) = \sum_{i=1}^{n} w_i f_i(s)$$

Uses properties of the environment (e.g., board position)

- Assumption: independence!

- Limitation:
  - May miss dependencies!
  - f(two bishops) > 2*f(one bishop)
  - f(bishop at end) > f(bishop at beginning)

Evaluation Functions are *Actually* Used

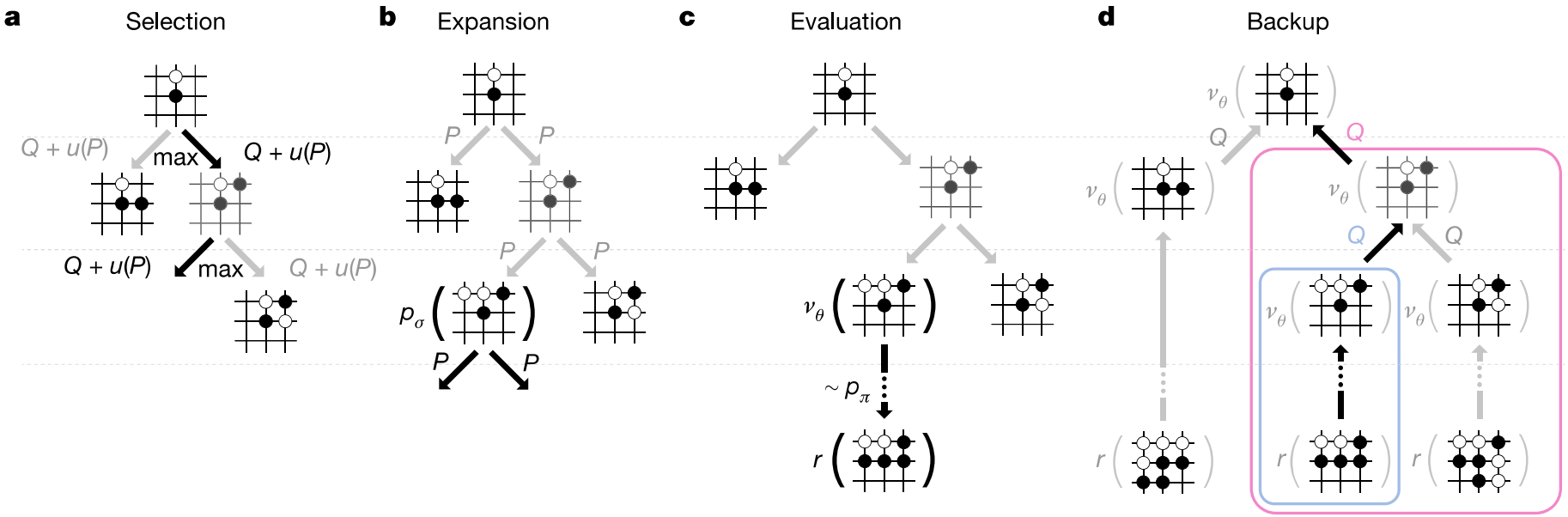


Figure 3 | Monte Carlo tree search in AlphaGo. a, Each simulation traverses the tree by selecting the edge with maximum action value Q, plus a bonus $u(P)$ that depends on a stored prior probability P for that edge. **b**, The leaf node may be expanded; the new node is processed once by the policy network p_σ and the output probabilities are stored as prior probabilities P for each action. **c**, At the end of a simulation, the leaf node is evaluated in two ways: using the value network v_θ; and by running a rollout to the end of the game with the fast rollout policy p_π, then computing the winner with function r. **d**, Action values Q are updated to track the mean value of all evaluations $r(\cdot)$ and $v_\theta(\cdot)$ in the subtree below that action.

Silver, David, Aja Huang, Chris J. Maddison, Arthur Guez, Laurent Sifre, George Van Den Driessche, Julian Schrittwieser et al. "Mastering the game of Go with deep neural networks and tree search." *nature* 529, no. 7587 (2016): 484.

Speed Up?

- **Problem we are addressing:** Alpha-beta pruning has to search all the way to terminal states…

- Let's use the evaluation function
  - Turn non-terminal nodes into terminal nodes
  - Use as a "cut-off" test

$$\text{H-MINIMAX}(s,d) = \begin{cases} \text{EVAL}(s) & \text{if CUTOFF-TEST}(s,d) \\ \max_{a \in Actions(s)} \text{H-MINIMAX}(\text{RESULT}(s,a), d+1) & \text{if PLAYER}(s) = \text{MAX} \\ \min_{a \in Actions(s)} \text{H-MINIMAX}(\text{RESULT}(s,a), d+1) & \text{if PLAYER}(s) = \text{MIN} \end{cases}$$

Important distinction:
How is an evaluation function different from a utility function?

Cutting off Search

- Challenges:
  - What is the proper cut-off point?
  - Short-term goal vs. long-term goal (e.g., chess…)

- How to address this:
  - Quiescent search

Forward Pruning

- Instead of traveling to tree depth (or cut-off depth) – cut a node immediately if it seems undesirable

- One method: Beam search
  - Consider only a “beam” of the n-best moves
  - Dangerous – best moves may be pruned away!

- ProbCut (version of alpha-beta)
  - Prune nodes probably outside of the (α, β) window

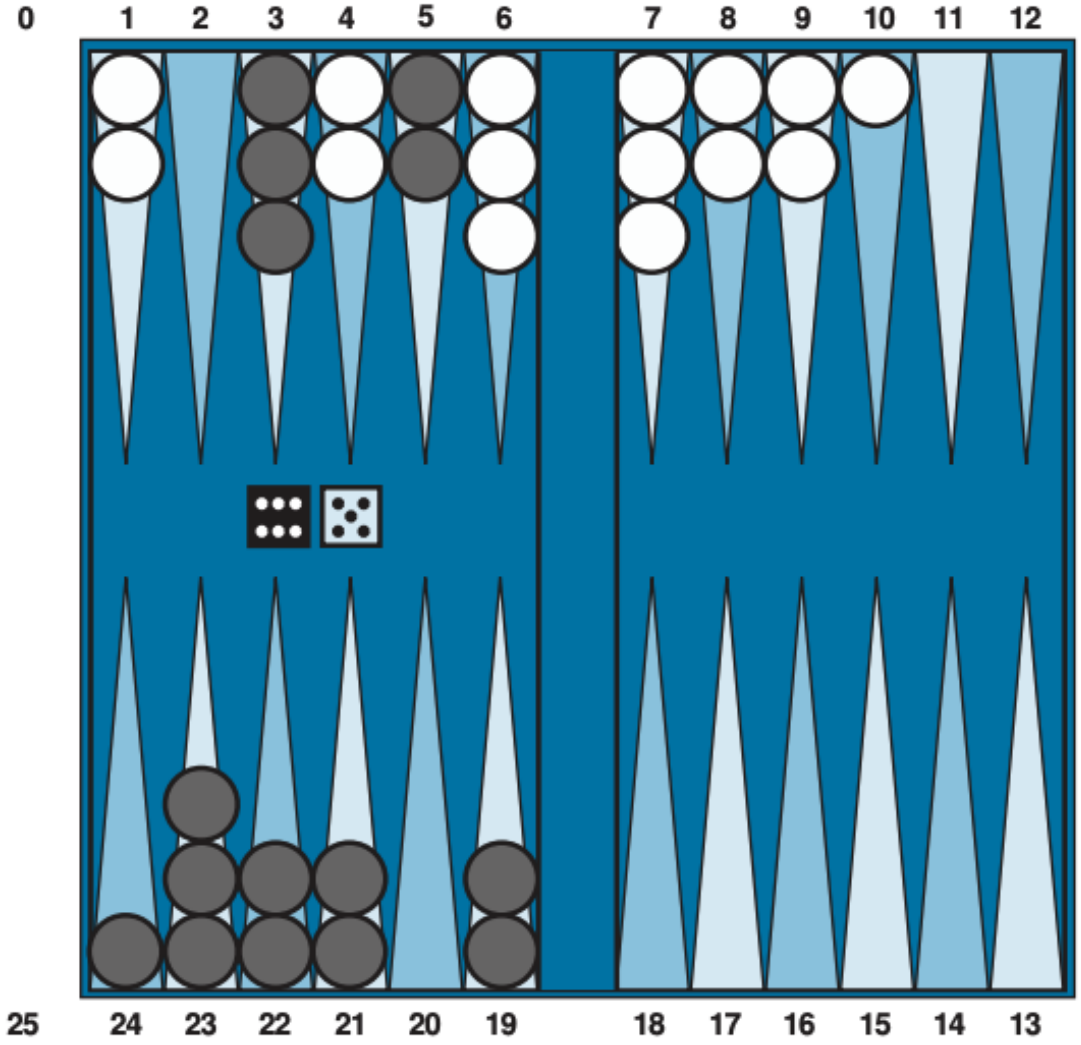
Stochasticity in Games (Adding another layer of complexity…)

Stochastic Games

- Uncertainty also relevant for games
- Can extend turn-playing model to include chance elements
- E.g., backgammon

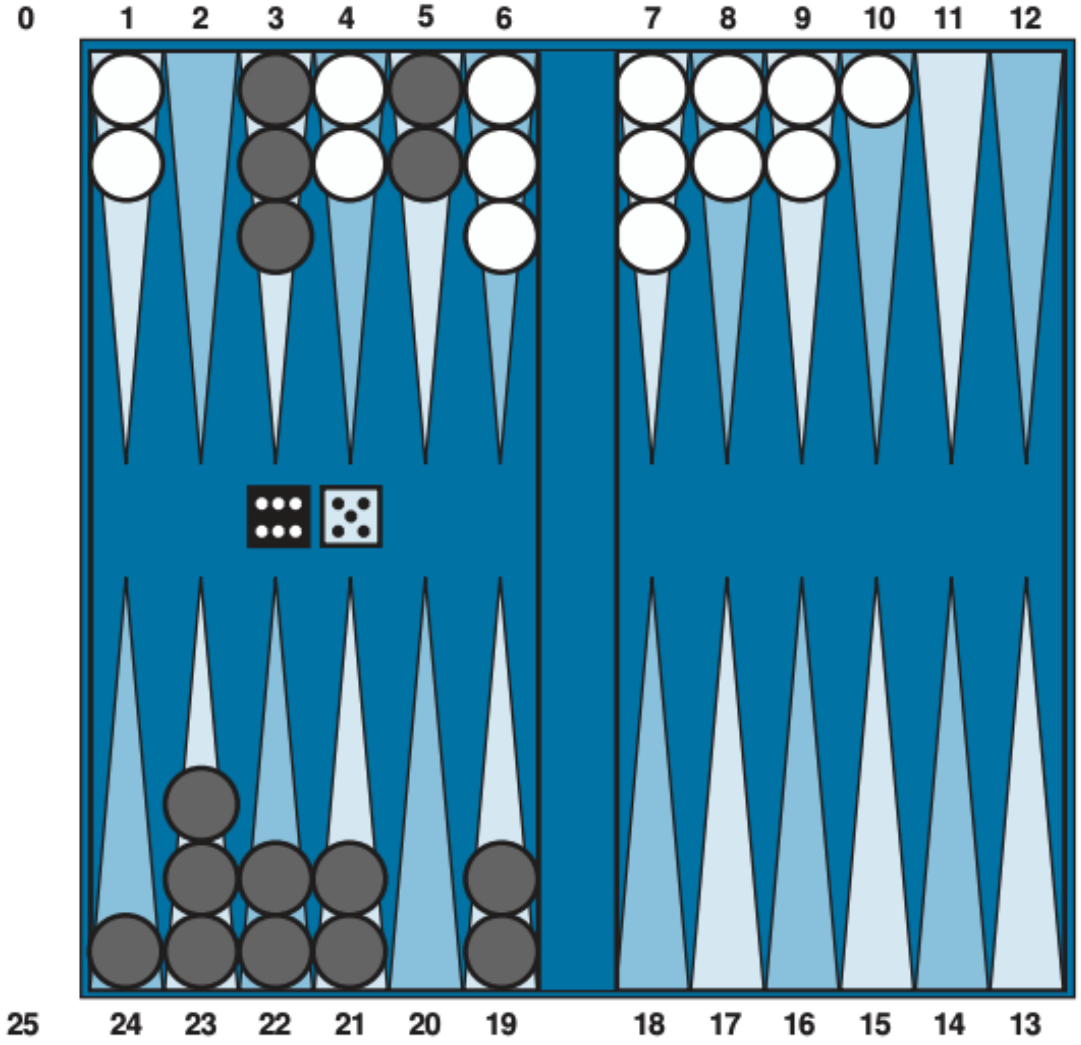
Backgammon in a Nutshell

- Goal:
  - Move all pieces off board
  - Black moves clockwise towards 25
  - White moves counterclockwise towards 0
- Movement
  - Based on roll of dice
  - Piece can move to any position unless multiple opponents are present
  - If one opponent- can capture and force restart



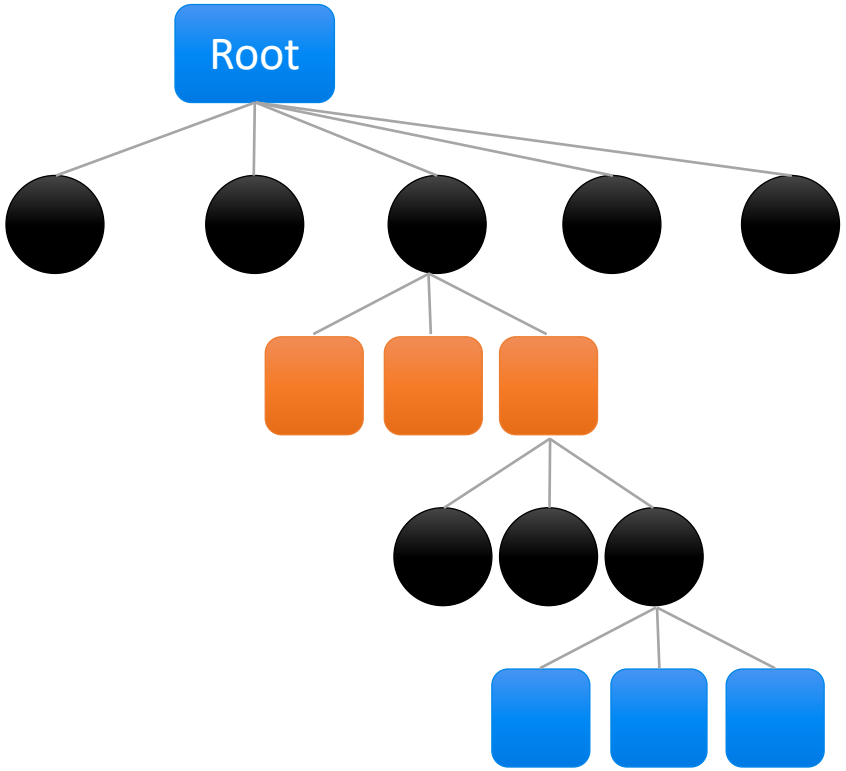
Why is this Hard?

- What white knows:
  - Its own legal moves
- What white doesn't know:
  - What black will roll
  - What black will do
- How we will deal with this:
  - Chance nodes!



Schematic Game Tree

- Partially the same:
  - Blue nodes: choose max
  - Orange nodes: choose min
- Black nodes are chance
  - Calculate the Expected Value
- Generalize to **Expectiminimax**



$$\textsc{Expectiminimax}(s) = \begin{cases} \textsc{Utility}(s) & \text{if } \textsc{Terminal-Test}(s) \\ \max_a \textsc{Expectiminimax}(\textsc{Result}(s,a)) & \text{if } \textsc{Player}(s) = \textsc{Max} \\ \min_a \textsc{Expectiminimax}(\textsc{Result}(s,a)) & \text{if } \textsc{Player}(s) = \textsc{Min} \\ \sum_r P(r)\textsc{Expectiminimax}(\textsc{Result}(s,r)) & \text{if } \textsc{Player}(s) = \textsc{Chance} \end{cases}$$

Stepping Back

- What if we knew the dice rolls?
  - Just minimax!
  - Time complexity: $O(b^m)$
- We must consider **all** possible dice rolls
  - n = number of dice rolls
  - Time complexity: $O(b^m n^m)$
- Can't leverage alpha-beta in the same way
  - There are no **likely** sequences (in terms of best play)